

Assignment 1: RTL and Testbench Generation

ECEN 689 | Fall 2026 | UIN 137008874

Task 1: Experiment setup

Assignment 1 uses a Python workflow to generate and verify four RISC-V building blocks: the ALU, register file, immediate extender and controller. Each YAML specification is evaluated with three prompt styles and two runs per style, giving 24 runs. Initial RTL passed all instructor samples in 20/24 runs; bounded repair increased this to 23/24.

All requests used TAMU AI Chat, the gpt-5.4 alias, an 8,192-token output limit and the same system instruction. Returned model: gpt-5.4-2026-03-05. Sampling used provider defaults. The experiment made 66 LLM calls including TB requests and repairs. Icarus Verilog 13 compiled SystemVerilog with -g2012.

The program in student_code/lab1.py reads the specification, generates RTL and a self-checking testbench, and invokes the provided Icarus helpers. Only instructor sample tests guide RTL repair. Each run permits at most three RTL repairs and two separate TB repairs, stopping early on success or identical RTL. The initial and repaired versions, prompts, full responses, model metadata and simulation logs are saved separately.

Commands followed

The following shell equivalent uses the same arguments, prompt styles and run names as the experiment launcher. Run it from the assignment root with the configured Python environment, API key and Icarus on PATH. Use unused output directories for a new experiment.

```
for block in alu regfile extend controller; do
  for style in direct constraints template; do
    for run in 01 02; do
      A1_STYLE="$style" python student_code/lab1.py \
        --spec "specs/$block.yaml" \
        --out-dir "results/experiment/$block/$style/run-$run" \
        --provider tamu
    done
  done
done
```

Each invocation generates RTL and a TB, scores the initial RTL before repairs, and preserves every attempt. Failed bounded runs remain in the results. The recorded launcher used four concurrent processes; the equivalent loop above runs sequentially.

Task 2: Prompt-style comparison

direct. The direct prefix asks for the specified implementation. Its ALU uses a compact case block; its register file clearly separates edge-triggered storage from reads. Initial full passes: 6/8; after repair: 8/8. The initial source differed between repeats for 3/4 blocks. Initial failures: controller. Interface validation rejects changed names, widths and port order before compilation. Samples do not measure synthesis quality.

constraints. The constraints prefix emphasizes widths, timing and defaults. Its controller visibly initializes every output and validates legal encodings before assigning controls, making defaults easy to audit. Initial full passes: 8/8; after repair: 8/8. The initial source differed between repeats for 3/4 blocks. Initial failures: none. Interface validation rejects changed names, widths and port order before compilation. Samples do not measure synthesis quality.

template. The template prefix fixes the interface. Its ALU names intermediate results clearly, but controller run 01 mixed continuous and procedural assignments to rd/rs1/rs2, causing an initial compile failure. Initial full passes: 6/8; after repair: 7/8. The initial source differed between repeats for 2/4 blocks. Initial failures: controller. Interface validation rejects changed names, widths and port order before compilation. Samples do not measure synthesis quality.

Task 2: All initial runs and repair outcomes

Block	Style	Run	Compile?	Initial tests	Remarks
alu	constraints	01	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
alu	constraints	02	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
alu	direct	01	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
alu	direct	02	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
alu	template	01	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
alu	template	02	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
controller	constraints	01	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
controller	constraints	02	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
controller	direct	01	yes	37/40	Best 40/40; 1 RTL repairs; TB pass
controller	direct	02	yes	37/40	Best 40/40; 1 RTL repairs; TB review
controller	template	01	no	not run	Best 37/40; 3 RTL repairs; TB review
controller	template	02	no	not run	Best 40/40; 2 RTL repairs; TB pass
extend	constraints	01	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
extend	constraints	02	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
extend	direct	01	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
extend	direct	02	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
extend	template	01	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
extend	template	02	yes	40/40	Best 40/40; 0 RTL repairs; TB pass
regfile	constraints	01	yes	39/39	Best 39/39; 0 RTL repairs; TB review
regfile	constraints	02	yes	39/39	Best 39/39; 0 RTL repairs; TB pass
regfile	direct	01	yes	39/39	Best 39/39; 0 RTL repairs; TB pass
regfile	direct	02	yes	39/39	Best 39/39; 0 RTL repairs; TB pass
regfile	template	01	yes	39/39	Best 39/39; 0 RTL repairs; TB pass
regfile	template	02	yes	39/39	Best 39/39; 0 RTL repairs; TB pass

Initial tests are scored before any repair. Every revised RTL has its own source, score and compiler/simulator logs in the submitted history. A successful simulation process is insufficient: success also requires one consistent nonzero count summary, zero failures and an exact PASS line without failure markers. Generated TB outcomes do not influence RTL repair.

Task 3: Final choices

alu: alu/constraints/run-01/rtl_00.sv. It passes 40/40 instructor samples and has a verified TB. It was chosen for the highest sample score; ties prefer a verified TB and an earlier RTL attempt. Final named files are in 02_hdl/final/.

regfile: regfile/constraints/run-02/rtl_00.sv. It passes 39/39 instructor samples and has a verified TB. It was chosen for the highest sample score; ties prefer a verified TB and an earlier RTL attempt. Final named files are in 02_hdl/final/.

extend: extend/constraints/run-01/rtl_00.sv. It passes 40/40 instructor samples and has a verified TB. It was chosen for the highest sample score; ties prefer a verified TB and an earlier RTL attempt. Final named files are in 02_hdl/final/.

controller: controller/constraints/run-01/rtl_00.sv. It passes 40/40 instructor samples and has a verified TB. It was chosen for the highest sample score; ties prefer a verified TB and an earlier RTL attempt. Final named files are in 02_hdl/final/.

Task 4: Testbench checks

alu: Arithmetic wraparound, logic, signed and unsigned comparisons, shifts and shift masking, plus the independently computed Sum output. Selected TB result: 1084/1084 comparisons. HDL comments and saved diagnostics describe the executed checks.

regfile: Both asynchronous read ports, x0 protection, writes across registers, write disable, falling-edge updates, synchronous reset priority and behavior between active edges. Selected TB result: 504/504 comparisons. HDL comments and saved diagnostics describe the executed checks.

extend: I/S/B/J/U immediate reconstruction, sign handling, fragmented fields and implicit low zero bits. Selected TB result: 134/134 comparisons. HDL comments and saved diagnostics describe the executed checks.

controller: Legal decode classes and control outputs, plus illegal instruction detection and suppression of state-changing controls. Selected TB result: 734/734 comparisons. HDL comments and saved diagnostics describe the executed checks.

TB expectations are derived from the YAML, using four-state comparisons so unknown values do not silently pass. The register-file prompt explicitly avoids assuming x1..x31 are initialized before reset and requests a manually driven clock. The final RTL is frozen while the TB is repaired; a failed TB cannot change already sample-correct RTL.

Failures and lessons

Observed failures and repairs

Controller direct prompts initially left controls inconsistent with illegal-case sample expectations. Template run 01 had a multiple-driver assignment error. Register-file TB failures included unsupported expression slicing and incorrect expectations around reset/clock transitions. These are distinct compiler, RTL-behavior and verification-code failure classes.

alu: 0/6 initial RTL runs did not fully pass; 1/6 runs needed TB revisions. Failed attempts and feedback are retained in the run records.

regfile: 0/6 initial RTL runs did not fully pass; 3/6 runs needed TB revisions. Failed attempts and feedback are retained in the run records.

extend: 0/6 initial RTL runs did not fully pass; 0/6 runs needed TB revisions. Failed attempts and feedback are retained in the run records.

controller: 4/6 initial RTL runs did not fully pass; 3/6 runs needed TB revisions. Failed attempts and feedback are retained in the run records.

Task 5: Wrap-up

The highest initial full-pass count was 8/8, achieved by constraints. The table separates initial performance from improvements caused by feedback. Two repeats per block/style support a small local comparison, not a statistically strong ranking. Repeated source hashes identify textual variation; they do not establish functional diversity. Results apply to the provided samples and interfaces.

For later processor integration, retain exact interfaces, explicit signedness, width checks and clock/reset scheduling in prompts. Keep an independent correctness oracle, preserve the first candidate, and feed concrete failing inputs and expected outputs into bounded repairs. For TBs, test only promised behavior and allow updates to settle before comparing. These habits are more defensible than assuming that a fluent response or a larger TB is automatically correct.

GitHub repository

https://github.com/kanishk-tamu/ECEN_689/tree/main/a1

The assignment code, generated RTL and testbenches, run records and report are stored in the a1 folder of the public ECEN_689 repository. The files can be viewed using the link above.